Course Project Writeup

Pokemon OOP Explorer

---

https://github.com/avih7531/pokemon-oop-explorer

Avi Herman (AJH773)

Date of Submission: April 27, 2026

Table of Work

(Please write x in the boxes to mention what each student achieved in this project)

	Avi	Student-2	Student-3
Project Description	x		
Uses Cases Diagram(s) and discription	x		
Sequence Diagrams	x		
Class diagram(s)	x		
Implementation	x		
Conclusion	x		

Table of Contents

System Analysis

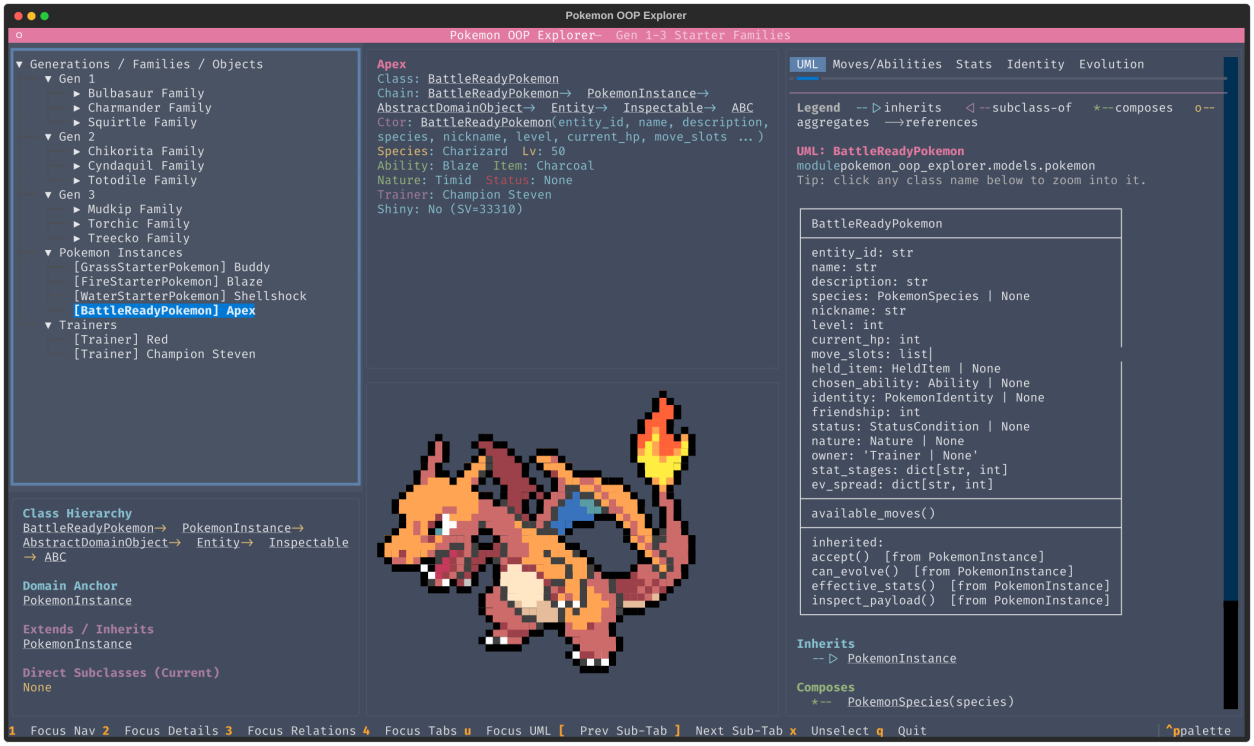


Figure: Application preview.

Written Project Description

General Description

Pokemon OOP Explorer is a terminal-based software system built to explore a well-scoped Pokemon domain model for Gen 1–3 starter families. Instead of focusing on a complete Pokedex or competitive battle simulator, this project focuses on object-oriented software engineering concepts: inheritance, composition, polymorphism, abstract interfaces, and clean separation of responsibilities. The application is implemented with Textual (TUI) and is organized in modular layers: models, services, introspection utilities, and UI presentation.

Goals and Benefits

The project goals are:

- Build a clear and maintainable object-oriented domain with behavior-rich classes, not just passive data structs.
- Provide an interactive inspector where users can navigate species, instances, and trainers, and inspect relationships in real time.

- Demonstrate important OOP patterns in practice: Visitor, Strategy (for graph edge detection), and Composite (for evolution rules).
- Produce UML artifacts that reflect the real implementation so design and code stay aligned.

The main benefits are educational and structural: the system makes OOP design decisions visible, testable, and reviewable, while keeping the domain scope small enough to reason about deeply.

Special Requirements

- **Performance:** The app runs locally on seeded/cached data and avoids live API dependency during normal usage, which keeps interactions responsive.
- **Interface constraints:** The UI requires a large terminal footprint for proper three-pane rendering and Unicode/ANSI compatibility.
- **Reliability:** Domain behavior is covered by automated tests (repository, evolution logic, identity, effects, class graph, and trainer operations), improving confidence in correctness.
- **Scope constraints:** Limited intentionally to Gen 1–3 starter lines and related trainer/instance identity/evolution flows.

Use Cases Diagram(s) and Use Case Description

The actor is a student/reviewer exploring and evaluating the domain model. Use-case and activity UML diagrams are provided under:

- docs/uml/use_cases/images/overview.png
- docs/uml/use_cases/images/uc-1_...png through uc-8_...png

The implemented use cases are:

- UC-1 Browse catalog
- UC-2 Inspect domain objects
- UC-3 View class relationships
- UC-4 Review moves and abilities

- UC-5 Analyze stats
- UC-6 Explore identity mechanics
- UC-7 Trace evolution
- UC-8 Preview sprites

System Design

Sequence Diagrams

Sequence diagrams are included for the main runtime interactions and logic paths, with method names aligned to implementation code:

- seq_1_app_startup.png
- seq_2_node_selection_refresh.png
- seq_3_zoom_class.png
- seq_4_evolution_check.png
- seq_5_trainer_party_update.png
- seq_6_identity_mechanics.png
- seq_7_stats_analysis.png

These sequences cover startup, selection rendering, UML focusing, evolution rule evaluation, trainer-party ownership updates, identity/shiny inspection, and effective stat computation.

Class Diagram(s)

The class diagram is provided in:

- docs/uml/class_diagram/images/domain_class_diagram.png

It models:

- Core contracts (`Inspectable`, `AbstractDomainObject`, `Evolvable`)
- Pokemon domain entities (`PokemonSpecies`, `PokemonInstance`, `Trainer`)
- Evolution hierarchy (`EvolutionRule` and concrete rule types)
- Visitor components (`DomainVisitor`, `ClassUsageVisitor`, `SummaryVisitor`)
- Application and services (`PokemonExplorerApp`, `DomainRepository`, `SpriteService`, graph builder/renderer)

Key required methods are explicitly represented as implemented: `accept`, `inspect_payload`, `can_evolve`, `effective_stats`, and `add_to_party`.

Conclusion

This project successfully demonstrates a focused OOP system with clear design boundaries and traceable UML documentation. The strongest outcomes are:

- A behavior-rich object model rather than a purely data-centric model.
- Practical use of OOP design patterns in a real codebase.
- Consistency between implementation and UML artifacts.
- Test-backed reliability for key domain mechanics.

Difficulties included balancing diagram completeness with scope, keeping sequence messages aligned with real method calls, and maintaining visual consistency across all UML outputs. Recommended next steps would be extending the same architecture to additional Pokemon families and adding more interaction-level tests for UI workflows.

Appendix

Artifact locations

- Use-case diagrams: docs/uml/use_cases/
- Class diagram: docs/uml/class_diagram/
- Sequence diagrams: docs/uml/sequences/
- Source code root: pokemon_oop_explorer/

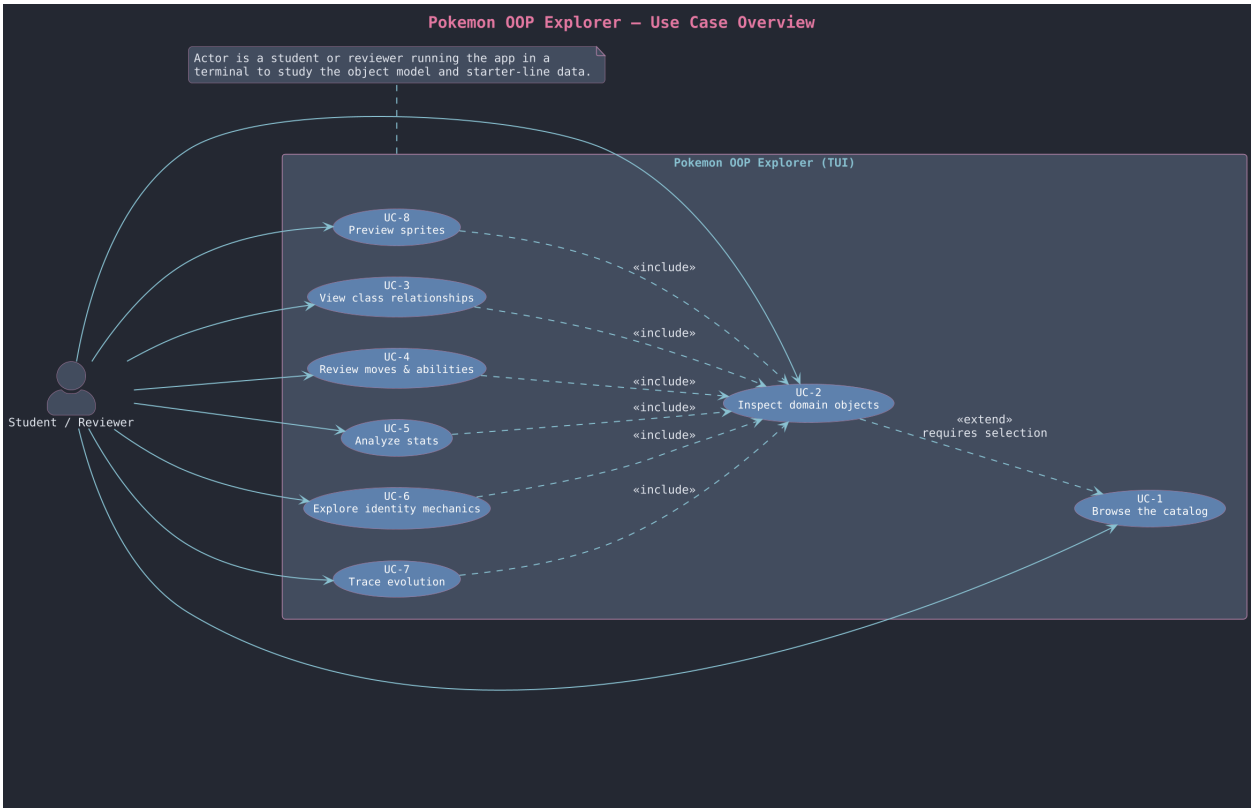
Diagram viewing note: Most UML diagrams are intentionally dense. Please zoom in when viewing this PDF. You can also open each image directly in its source folder under `docs/uml/`.

Coverage note: The codebase contains **194** defined functions/methods in `pokemon_oop_explorer/`. Using all UML source files in `docs/uml/**/*.puml`, **81** definitions are represented explicitly and **113** are not explicitly represented. This is intentional: the diagrams emphasize architecture-critical behavior rather than every helper.

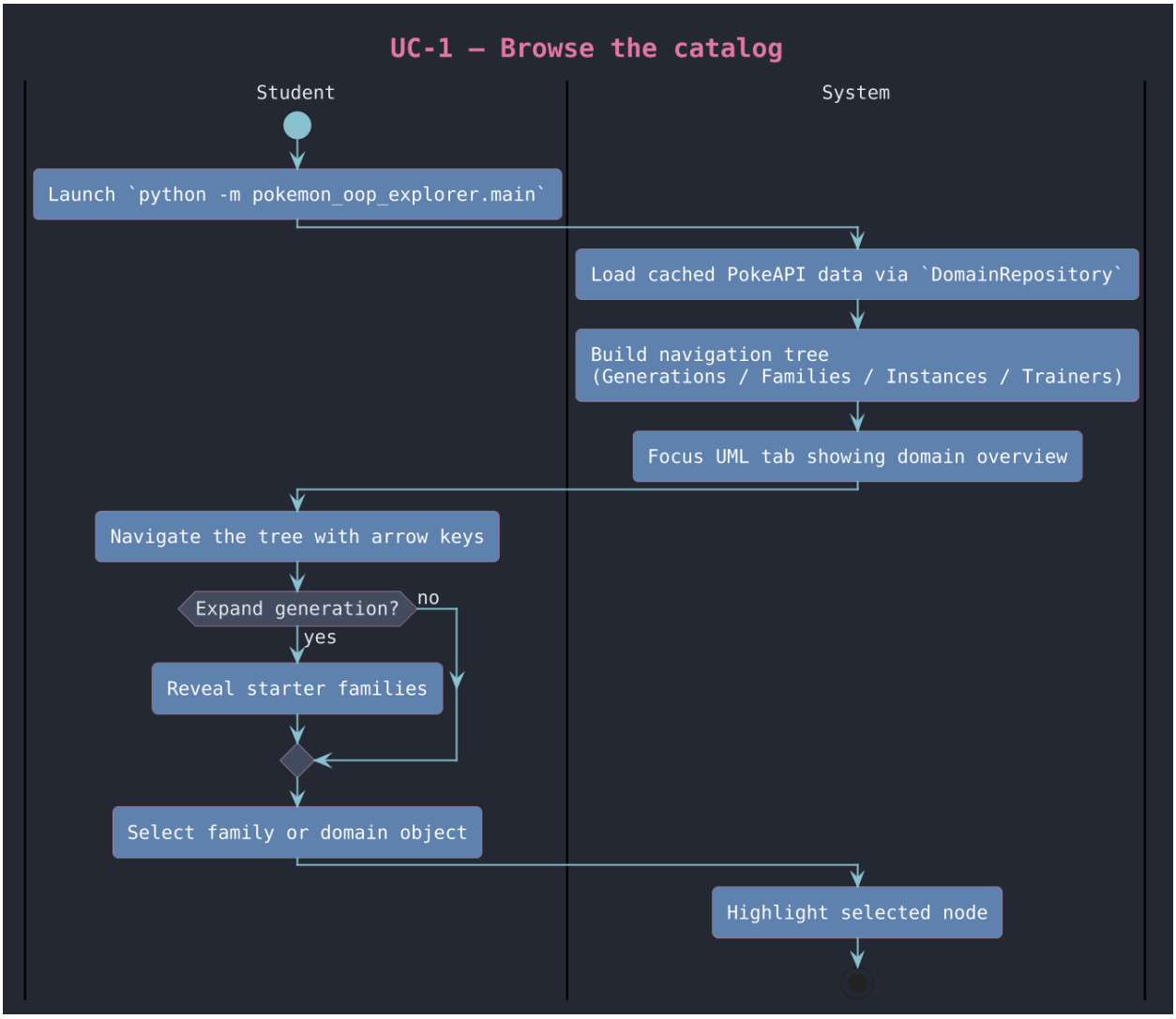
UML Diagram Gallery

Use Cases

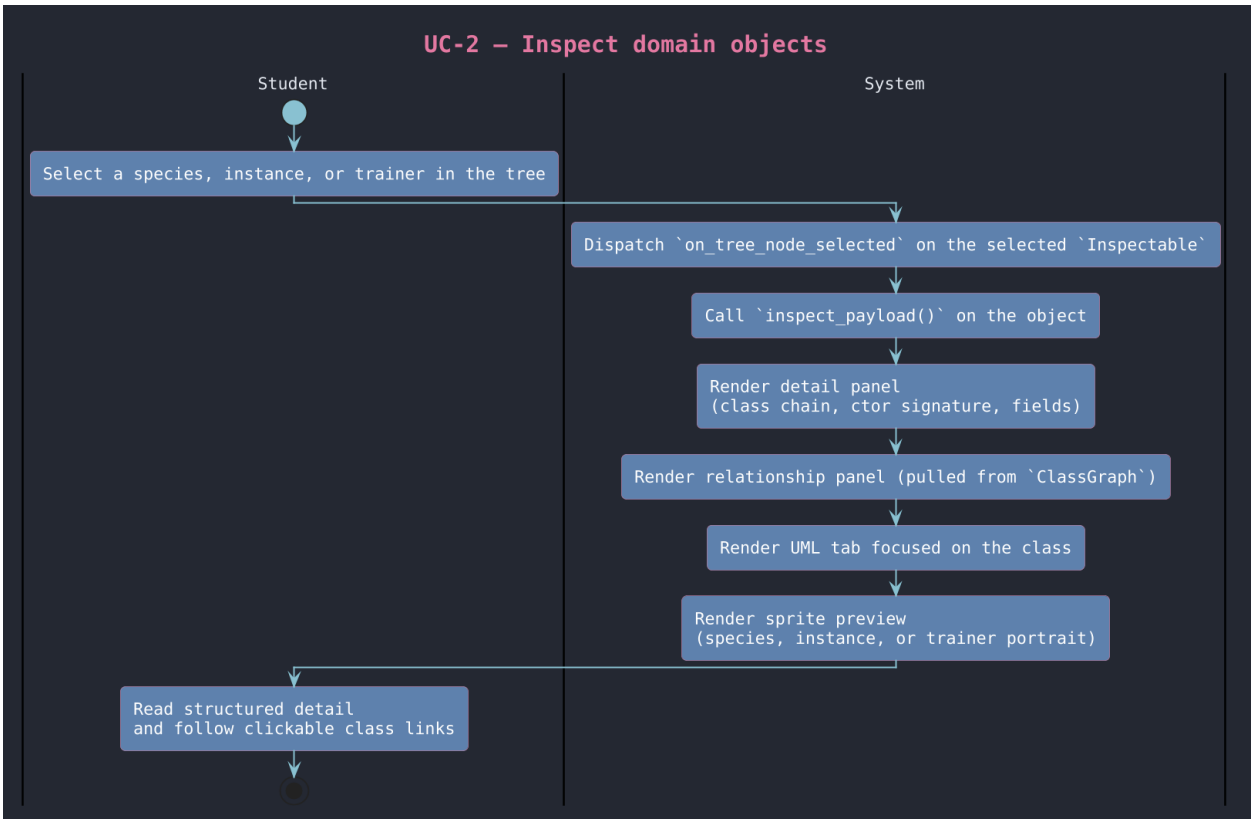
Zoom recommended for these diagrams. Source folder: `docs/uml/use_cases/images/`.



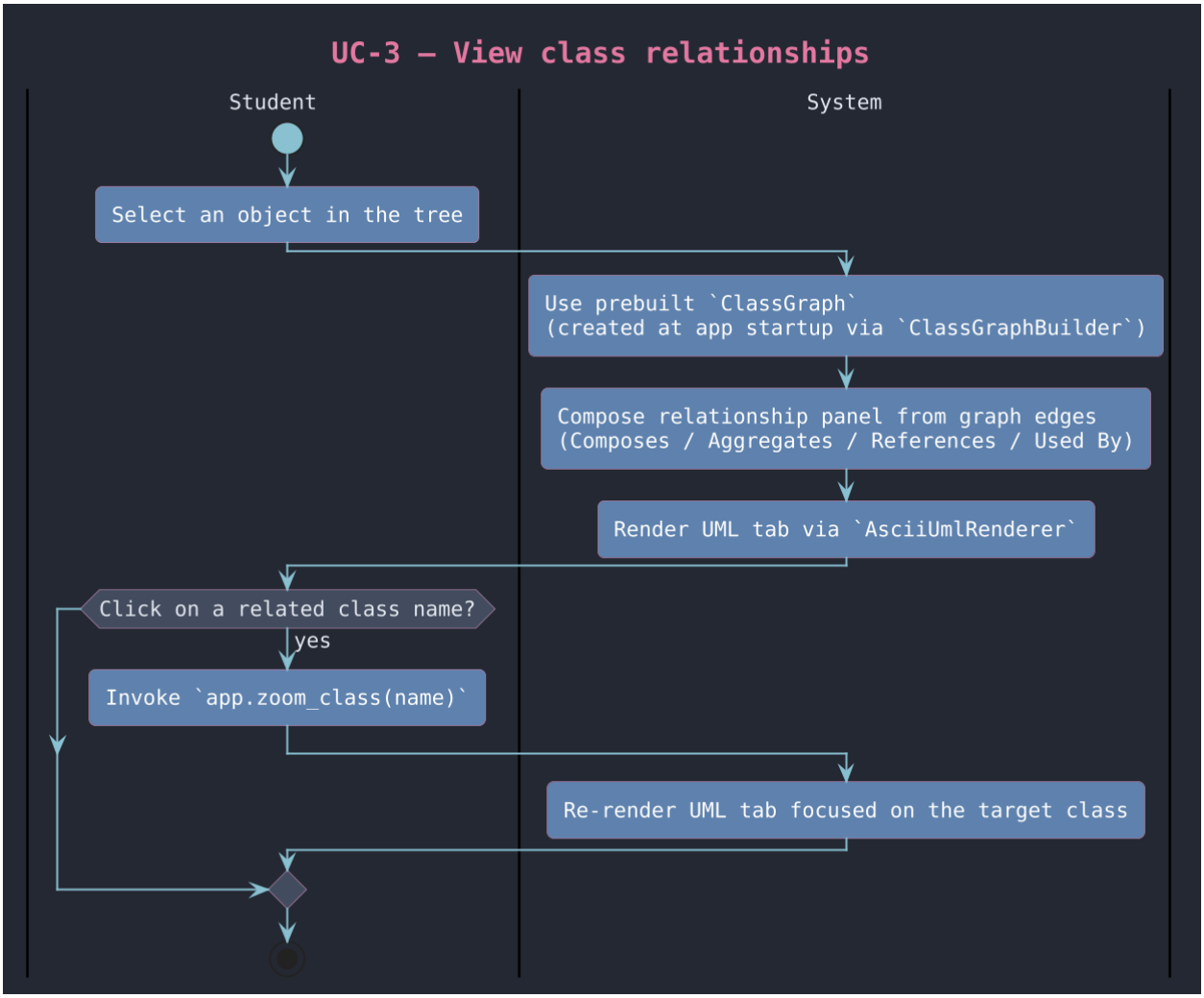
Overview use-case diagram.



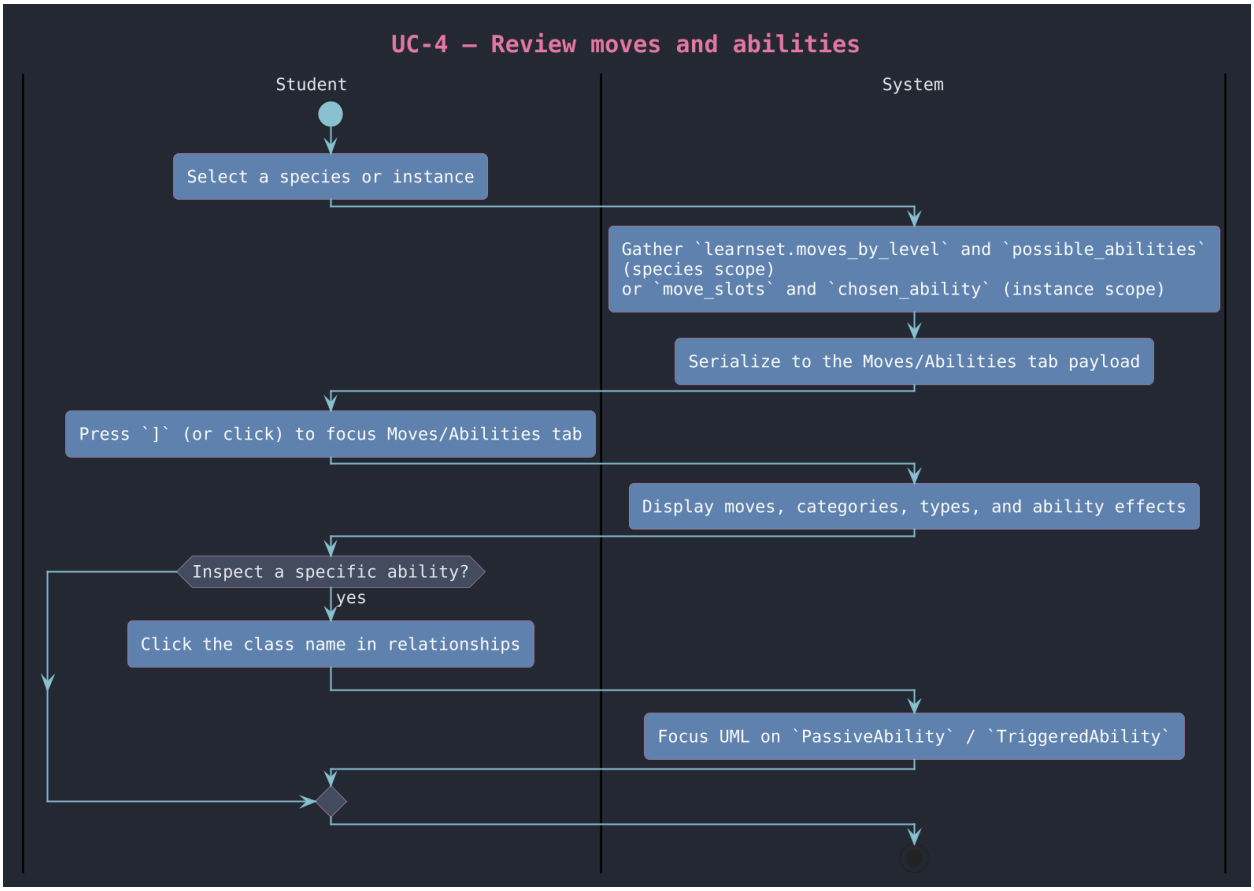
UC-1 Browse catalog (zoom recommended).



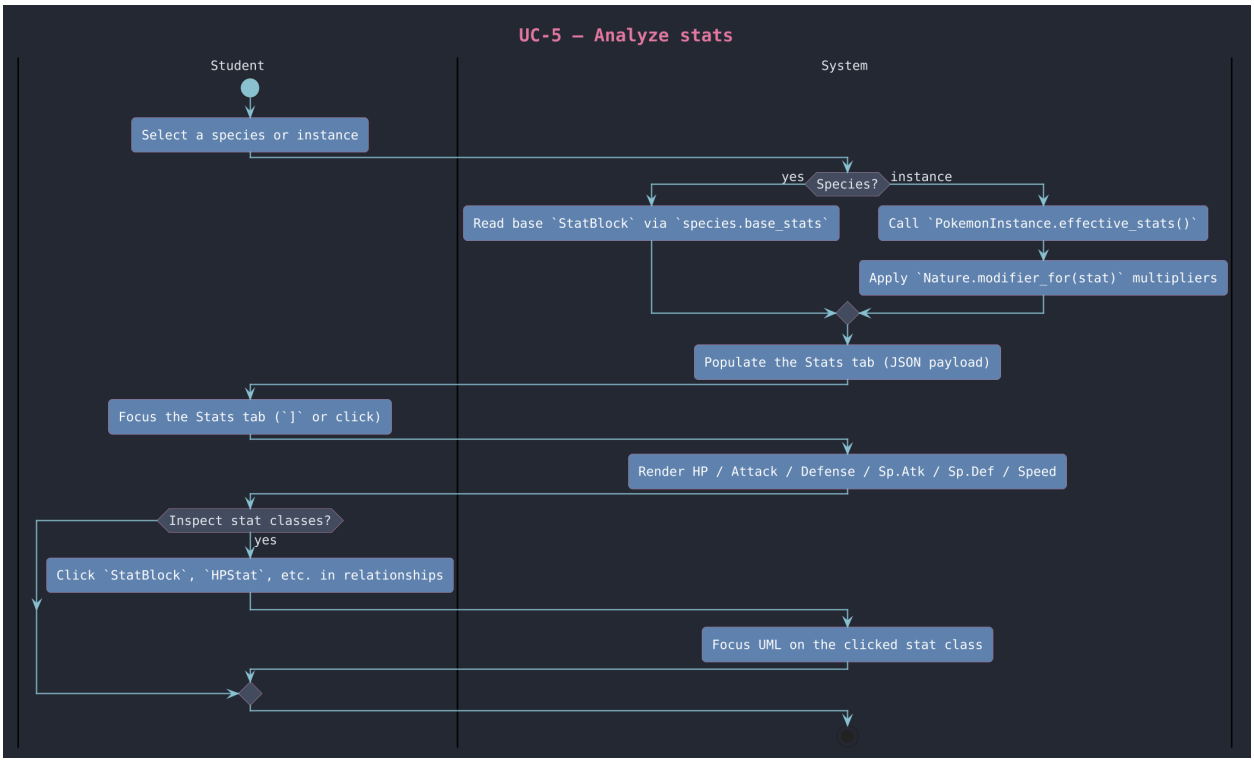
UC-2 Inspect domain objects (zoom recommended).



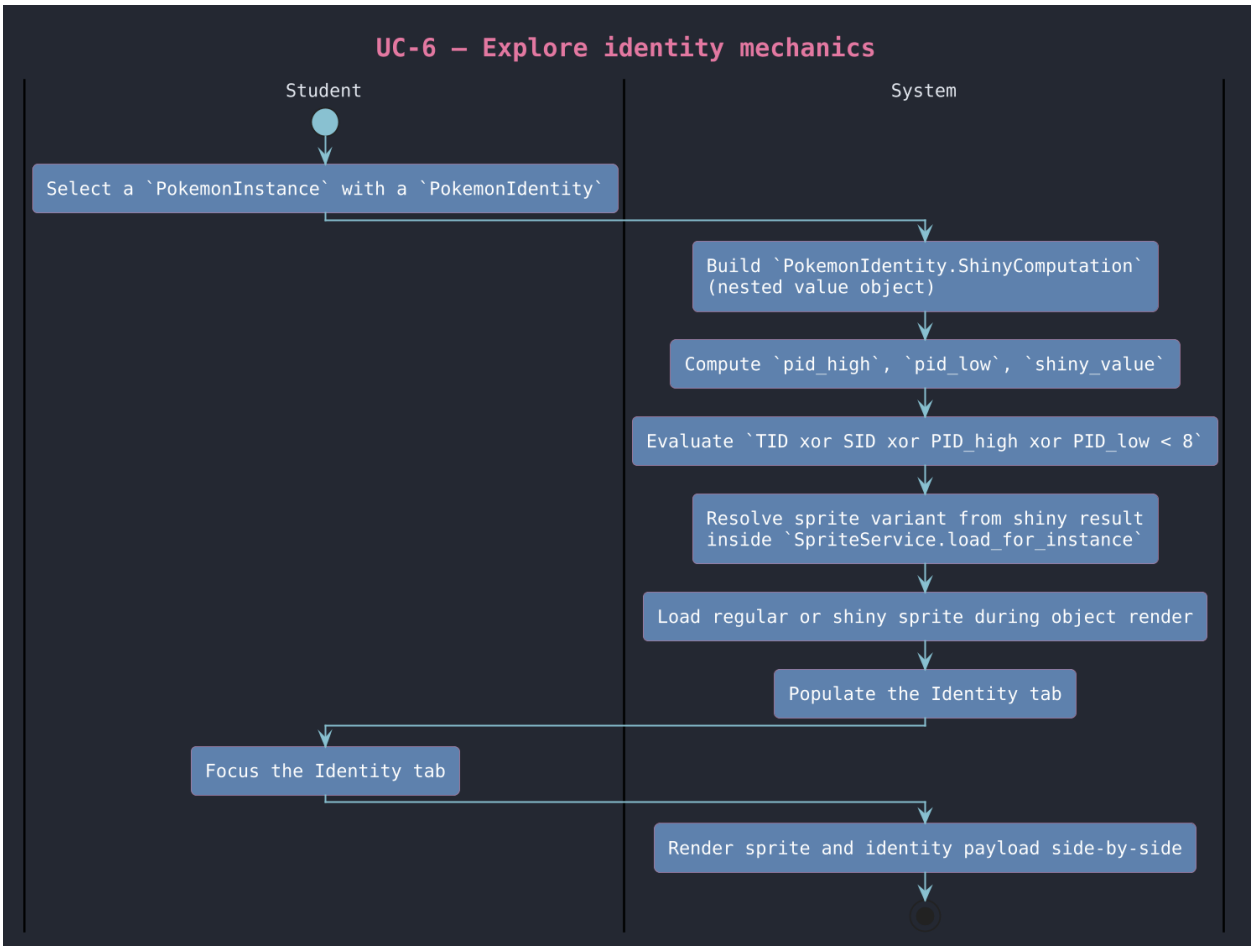
UC-3 View class relationships (zoom recommended).



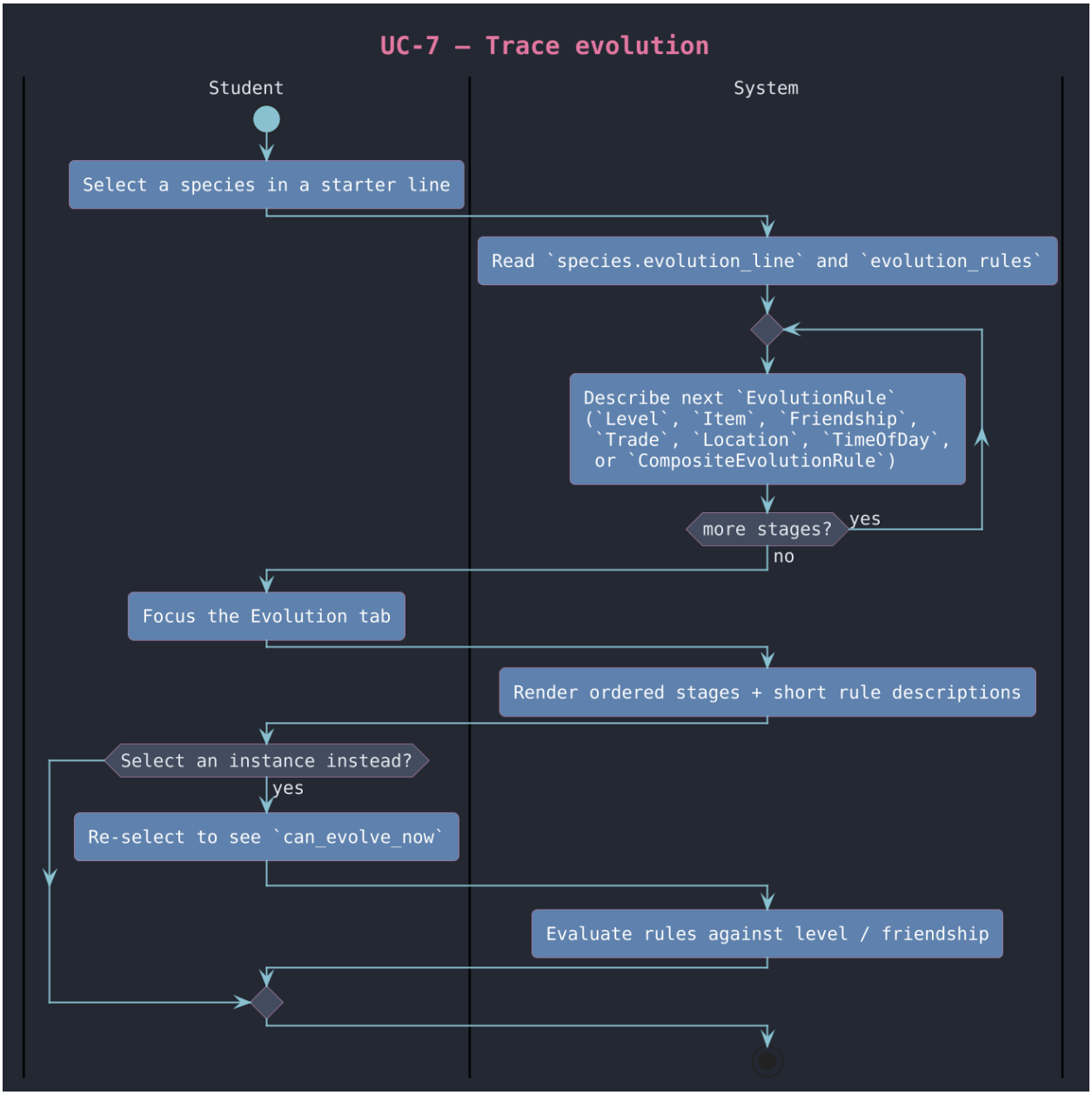
UC-4 Review moves and abilities (zoom recommended).



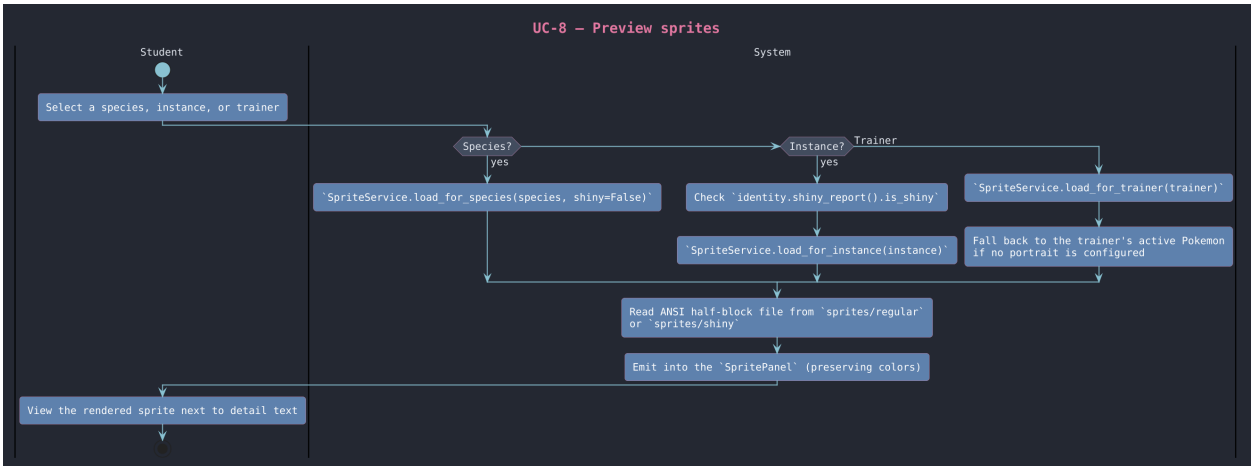
UC-5 Analyze stats (zoom recommended).



UC-6 Identity mechanics (zoom recommended).



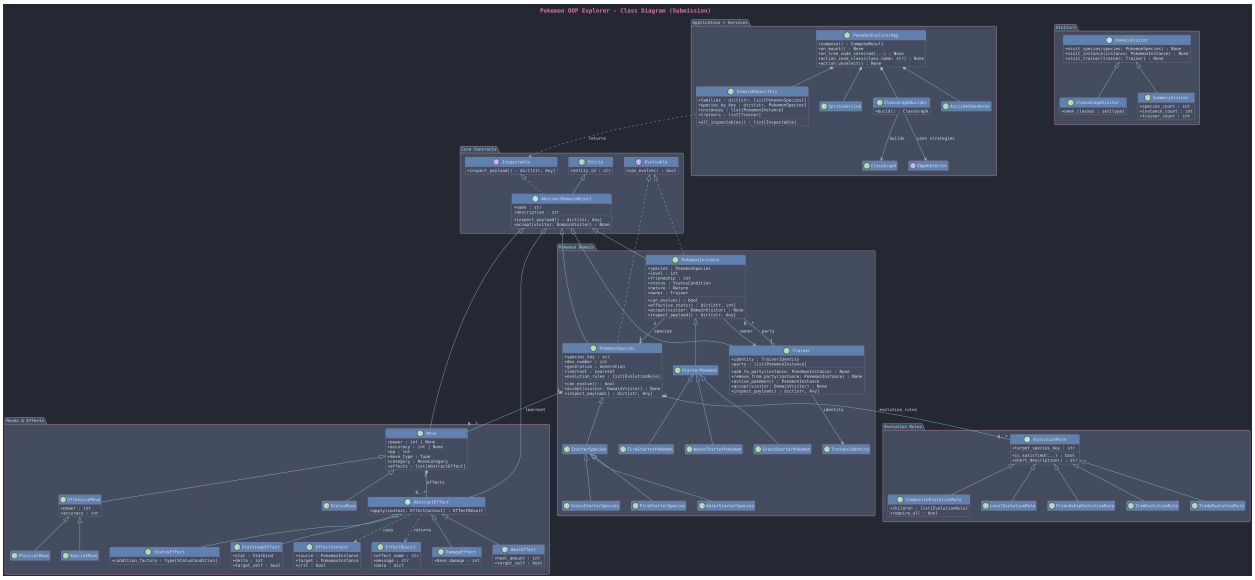
UC-7 Trace evolution (zoom recommended).



UC-8 Preview sprites (zoom recommended).

Class Diagram

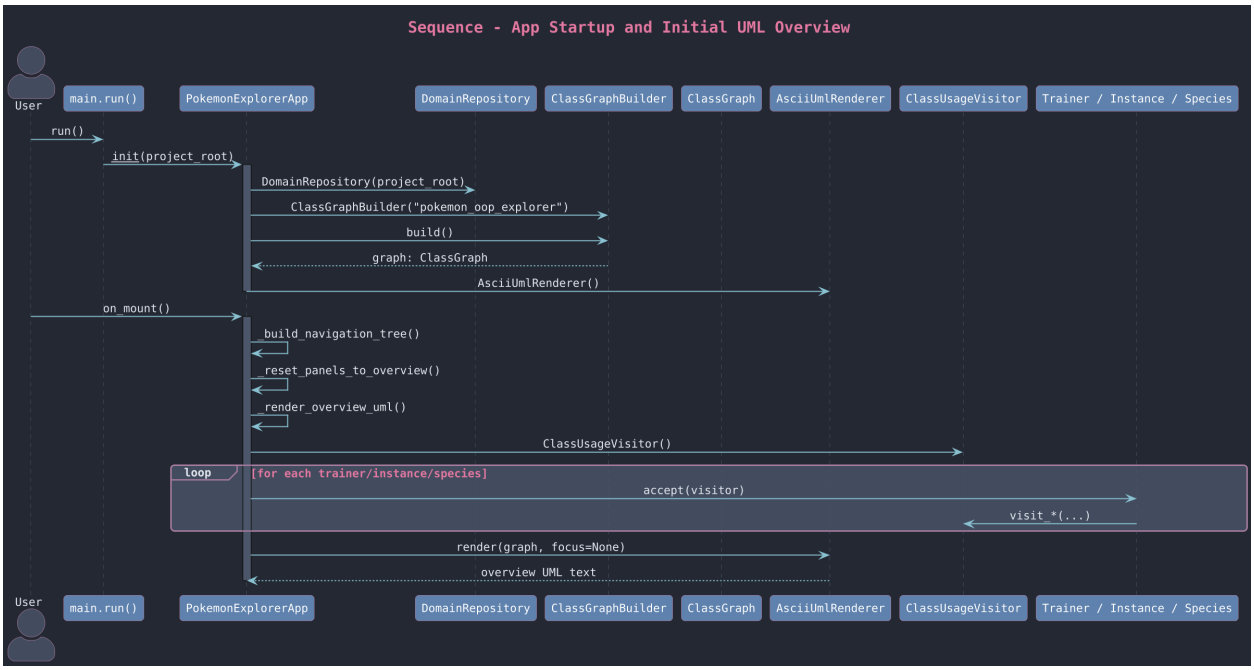
Large detailed diagram; zoom strongly recommended. Source: `docs/uml/class_diagram/images/`.



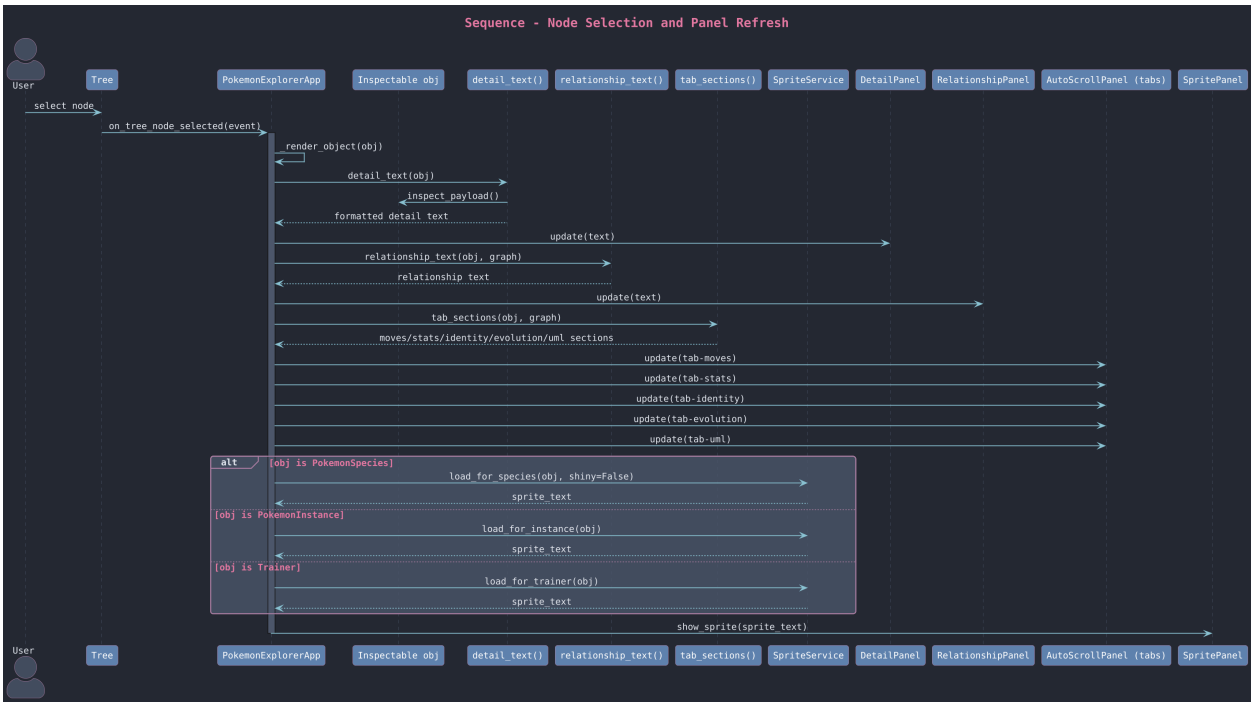
Implementation-aligned class diagram.

Sequence Diagrams

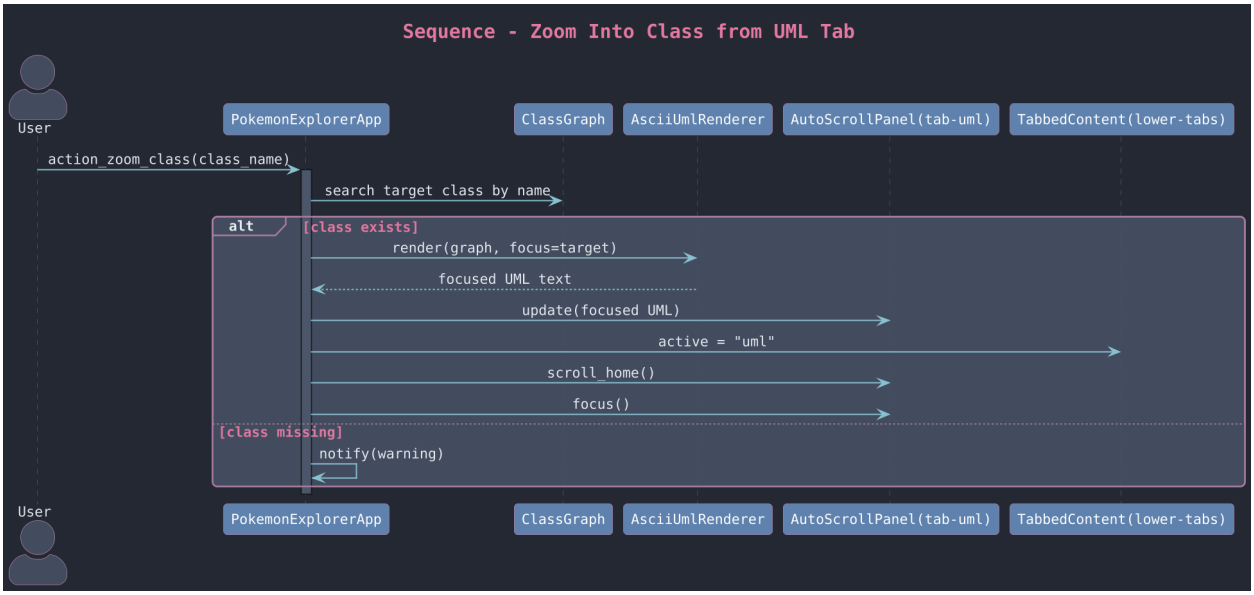
Most sequence diagrams are dense; zoom in or open the originals in `docs/uml/sequences/images/`.



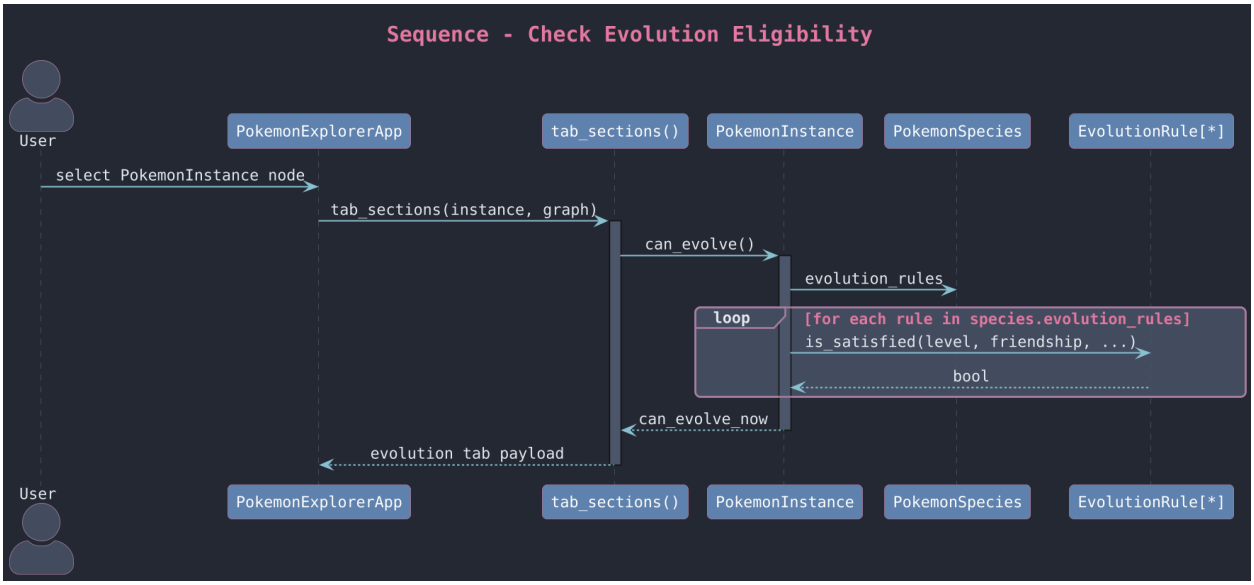
Sequence 1: app startup and initial UML overview (zoom recommended).



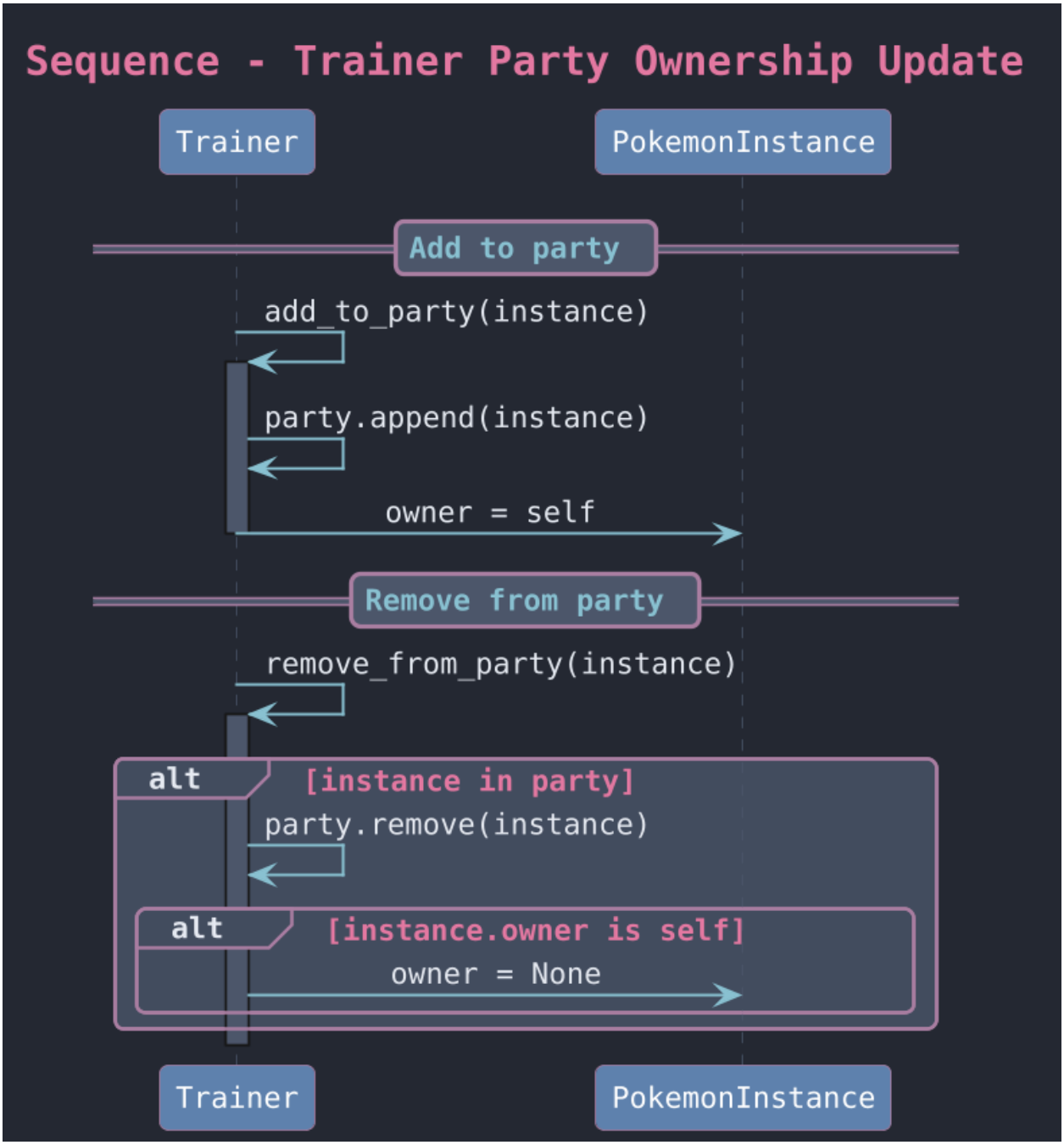
Sequence 2: node selection and panel refresh (zoom recommended).



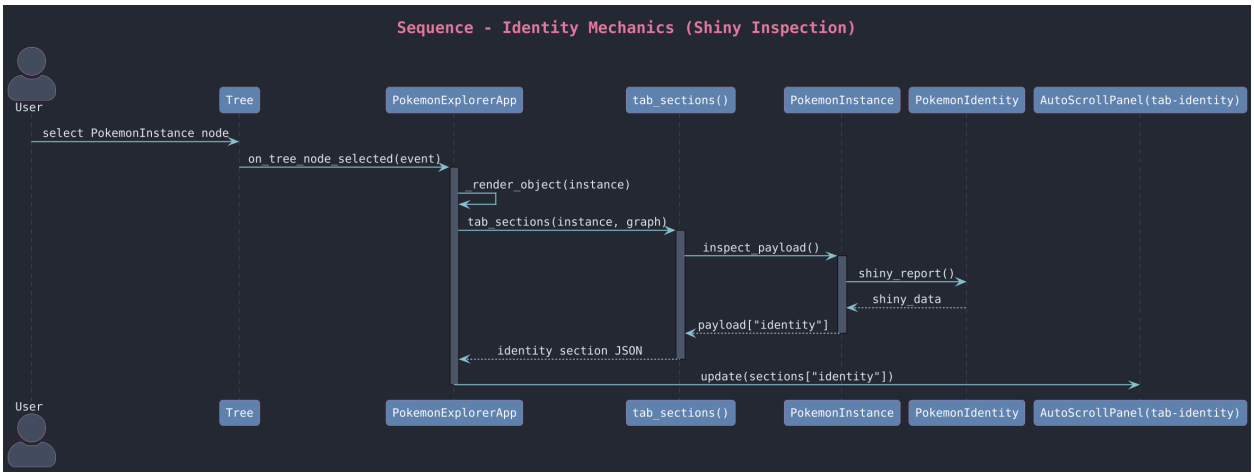
Sequence 3: zoom class interaction.



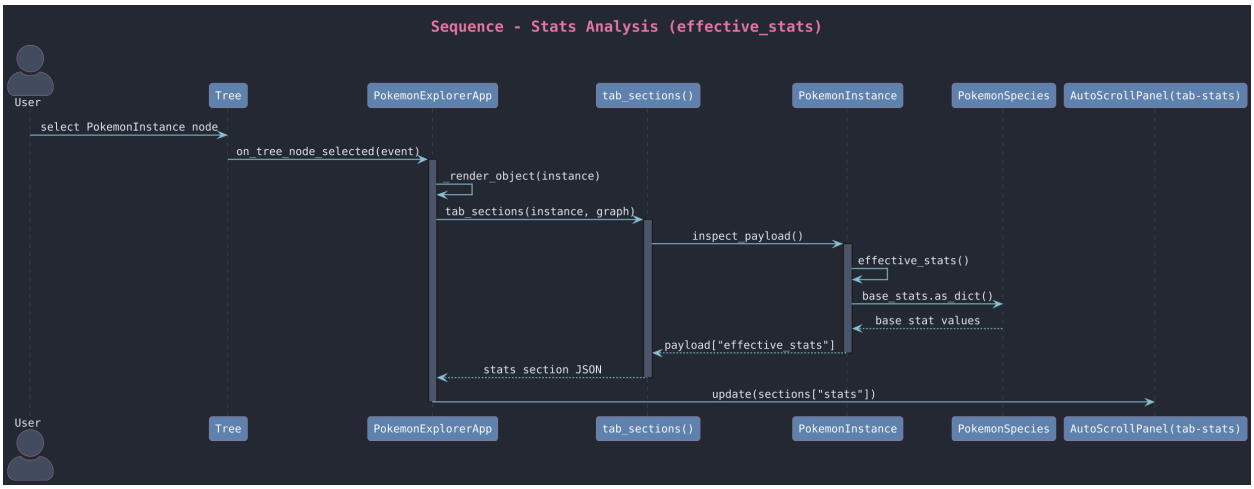
Sequence 4: evolution eligibility check.



Sequence 5: trainer party update (zoom recommended).



Sequence 6: identity mechanics (zoom recommended).



Sequence 7: stats analysis (zoom recommended).